Multimedia Tools and Applications (2025) 84:5435–5455
https://doi.org/10.1007/s11042-024-20523-1

Optimized Convolutional Neural Network at the IoT edge for image detection using pruning and quantization

Soumyalatha Naveen[1] · **Manjunath R Kounte**[2]

Received: 20 February 2024 / Revised: 3 October 2024 / Accepted: 12 December 2024 /
Published online: 26 December 2024
© The Author(s) 2024

Abstract
Most real-time computer vision applications heavily rely on Convolutional Neural Network (CNN) based models, for image classification and recognition. Due to the computationally and memory-intensive nature of the CNN model, it's challenging to deploy on resource-constrained Internet of Things (IoT) devices to enable Edge intelligence for real-time decision-making. Edge intelligence requires minimum inference latency, memory footprint, and energy-efficient model. This work aims to develop an energy-efficient deep learning accelerator using a 3-stage pipeline: Training, Weight-pruning, and Quantization to reduce the model size and optimize the resources. First, we employ YOLOv3, a CNN architecture to detect objects in an image on the trained data. In addition, a sparse network of YOLO has been created by using pruning, which helps to improve the network's performance and efficiency by reducing the computational requirements. Finally, we utilize 8-bit quantization to reduce the precision of the weights and activations, in a neural network. The evaluation of our proposed model shows that combining pruning and 8-bit quantization improves the efficiency and performance of the model. While pruning shows a decline of 80.39% in model parameters. The combination of 8-bit quantization results in an improvement in inference latency by 22.72% compared to existing SQuantization approach and a reduction of energy consumption by 29.41%.

Keywords Convolutional Neural Network · Edge · Pruning · Quantization · Resource optimization

1 Introduction

Edge computing [1] refers processing of data at the edge of a network rather than sending it to a central location for processing. This can be beneficial in cases where real-time processing is required, or the data needs to be processed locally due to security or privacy concerns. CNNs are a type of machine learning algorithm that is particularly effective at image classification

✉ Soumyalatha Naveen
soumyalatha.naveen@manipal.edu

[1] Department of Computer Science and Engineering, Manipal Institute of Technology Bengaluru, Manipal Academy of Higher Education, Manipal, India

[2] School of Electronics and Communication Engineering, REVA University, Bangalore, India

and object recognition tasks. The combination of edge computing and CNNs [2, 3] can be used in various applications [4], including facial recognition, object detection, facial emotion detection [5] and autonomous vehicles. CNN is used for honey quality evaluation [6] wherein pre-processing is carried out for the video of the honey sample. Further. there are many ways video can be split into frames [7] for easier pre-processing of the data. CNN plays major role in genomics data analysis [8] for identifying the genes and understanding the impact of change in genome.

GPS-based Intelligent transport systems provide real-time position of persons and vehicles [9] having smart devices. However, in addition to GPS analyzing the image strengthens the Intelligent transport system. For instance an autonomous vehicle could use edge computing to process sensor data and make real-time decisions about its surroundings, while a CNN could identify and classify objects in the environment as shown in Fig. 1. CNN enable object detection in driverless cars. They work by analyzing an input image and identifying patterns and features within the image. These patterns and features are then used to classify the image and make predictions. However, deploying the model on IoT devices is challenging due to the resource-constraint nature of IoT devices and the vast memory requirement of CNN models. To optimize the performance of the object detection model in driverless cars [10],it is essential to use a compact and efficient model that can run on the limited hardware resources of a vehicle. Hence, various techniques, such as pruning, quantization [11–14], low-rank decomposition [15], etc., need to be adopted to design a compact and efficient CNN model. With this, it is possible to create a CNN model that is small enough to run on a driverless car while maintaining good accuracy and performance.

YOLOv3 is a specific version of a CNN designed for object detection [16], using a combination of convolutional layers and anchor boxes to detect objects in an image. Using YOLOv3 in edge computing [17] can be beneficial because it allows real-time object detection without sending data to a central location for processing. This can be particularly useful in applications such as security and surveillance, where real-time processing is essential. However, it is important to consider the limitations of edge computing, such as the available computing power and the need for regular updates to the model to maintain accuracy.

The two common approaches for simplifying CNN are pruning and quantization. Pruning is a process of reducing the number of parameters in a neural network by removing redundant or unnecessary connections and parameters. This can be done to reduce the size of the network, improve performance, and reduce computational costs. In the case of YOLOv3 and the COCO dataset, pruning the network may be useful in order to deploy the model on devices with limited resources. One more approach 8-bit quantization is a process of reducing the precision of the weights and activations in a neural network. Using 8-bit integers instead of floating point numbers reduces the size of the network and the computational costs. Combining pruning and 8-bit quantization can further improve the efficiency and performance of the

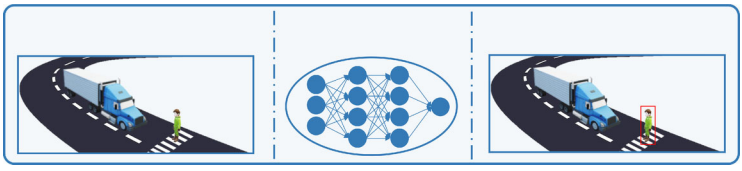


Fig. 1 Ultra-low latency real time driverless car scenario: Autonomous vehicles rely on various technologies to navigate and make decisions on the road. Edge computing is helpful for driverless cars because it allows them to quickly process and analyze large amounts of data in real time without sending it back and forth over a network

YOLOv3 model on the COCO dataset. However, it is important to carefully evaluate the impact on model accuracy and ensure that the trade-offs are acceptable.

For distributed edge device computation, inference latency, energy consumption [18] and communication overhead depend on the number of parameters in the model. Our primary focus is energy consumption, as most embedded or mobile devices run on batteries and require a large amount of memory and bandwidth to retrieve the neural network weights and perform convolution operations. It is evident that floating point add process takes three orders of magnitudes of add operation [19].

Hence, the power-hungry applications based on the deep neural network make deployment harder on resource constraint IoT devices. In AI-enabled IoT devices, energy consumption is less if the computing and data transmission is less. Hence to reduce the computing and data movement and consequently lower energy usage, many researchers suggest developing a more petite model through pruning and quantization by representing data in lower bits such as 8-bit, 4-bit, 2-bit, and 1-bit. Retraining after quantization helps to bring up accuracy. Compression, layer partitioning, weight sharing, partition scheduling, and knowledge distillation are the common approaches for optimizing the hardware and software in Edge intelligence.

Weight pruning and quantization [20] is a model compression technique that has received attention as one of the most efficient options to reduce the intensive computation and memory usage without compromising accuracy. We aim to reduce the storage and energy consumption of a CNN-based IoT Edge computing model.

Pruning or quantization has shown excellent potential on relatively smaller datasets such as MNIST or CIFAR for image classification tasks. As we would like to work on larger dataset and deeper network ; hence in continuation to our previous work [21], we intend to design a compact optimized Deep learning model considering larger dataset and heavy deep learning model.

The ***objective of this work is threefold and our contributions are mentioned below***.

- We **propose a unified DNN pruning framework** to perform pruning based on the magnitude of the weights of a trained model, to improve the model accuracy by removing the redundant information.
- **We formulated the pruning and 8-bit quantization with K-means clustering algorithms** to reduce the model further and design a compact energy-efficient sub-model of the pre-trained model.
- **To evaluate the performance of our proposed framework joint pruning and quantization** by considering the trade-offs between model performance, mean Average Precision, size, model parameters and energy consumption.

Results have surpassed state-of-the-art pruning and quantization and achieve better results. This makes deep learning models more energy efficient, enabling deploying the models on resource-constrained devices such as smartphones or edge devices. In addition, optimized model results in faster prediction, require less communication overhead and can easily be deployed on embedded devices in real-time. These factors motivated to focus on developing a novel approach to design a smaller pre-trained network for object detection applications.

The Table 1 shows the list of abbreviations in alphabetic order. The remaining parts of the article are organized as follows. Section 2 briefs Related work. Section 3 discusses the proposed system. Section 4 presents the experimental setup and results. Finally, we conclude the paper in Section 5.

Table 1 List of important abbreviations in alphabetical order

Acronym	Definitions	Acronym	Definitions
AI	Artificial Intelligence	ASIC	Application-Specific Integrated Circuits
CMOS	Complementary metal-oxide semiconductor	CNN	Convolutional Neural Networks
COCO	Common Objects in Context	CPU	Central processing unit
CUDA	Compute Unified Device Architecture	DNN	Deep Neural Network
DRAM	Dynamic random access memory	FFT	Fast Fourier transform
FLOPS	Floating point operation per second	FPGA	Field Programmable gate arrays
FTP	Fused Title partitioning	GPU	Graphics processing unit
HOG	Histogram of Oriented Gradients	IG	Information Gain
ILSVRC	ImageNet Large Scale Visual Recognition Challenge	IoT	Internet of Things
KD	Knowledge Distillation	L1-norm	L1 Regularization
mAP	Mean Average Precision	MCC	Mobile Cloud Computing
MEC	Mobile Edge Computing	MI	Mutual information
NN	Neural Network	pJ	petaJoule
QoE	Quality-of-Experience	RAN	Radio Access Network
SGD	Stochastic Gradient Descent	SPP	Spatial pyramid pooling
SRAM	Static Random Access Memory	SSL	Structured Sparsity Learn
VGG	Visual Geometry Group	YOLO	You Only Look Once

2 Related work

DNNs are the heart of many state-of-art of computer vision applications. A lot of computation power and storage is required to execute deep neural networks. However, the vast demand for computing resources significantly hampered the implementation of massive deep neural networks on devices having limited resources. Therefore, much research focuses on compressing the large-scale DNN to minimize storage and computing expenses to lower computation costs. Hence in our literature survey we focus on CNN based model compression approaches and followed by state of art of designing and energy efficient CNN model.

2.1 Model compression and acceleration approaches

Table 2 presents various model compression approaches used in the literature. Compression, layer partitioning, weight sharing, partition scheduling, and knowledge distillation are the common approaches for optimizing the hardware and software in Edge intelligence [22]. The structured Sparsity Learning (SSL) approach enables regularising the structures of DNNs

Table 2 Various Model compression and acceleration approaches

Approach	Description	Parameters
Pruning [23–25]	Remove the unimportant parameters in the model	Inference latency and Memory footprint
Model quantization [26–28]	Using less bits to indicate each weight for model compression	Reduces the memory footprint
Knowledge distillation [29, 30]	Teacher-student model, where in large Teacher model's knowledge is distributed to smaller model	Inference latency, memory footprint
Fused Tile partitioning [21, 31]	Partitioning the layers as independent tasks to distribute to various IoT devices	Inference latency, memory footprint

(filters, channels, filter shapes, and layer depth) and producing hardware-friendly structured sparsity of DNN for faster DNN evaluation, in turn, to lower computing costs. The large storage requirement of deep learning models due to over-parametrization enforces pruning to reduce the memory footprint and to speed up the convolutional layer computations.

Pruning reduces the complexity of a neural network by removing unnecessary or redundant connections and neurons. This can improve the efficiency and performance of the network without significantly affecting its accuracy. Creating a sparse network using pruning for YOLOv3 COCO dataset involves identifying and removing unnecessary or redundant connections within the network. This can be done through a variety of methods such as manual pruning, weight pruning, and neuron pruning. Manual pruning involves manually identifying and removing connections that are not contributing to the performance of the network. This can be time-consuming and may not always result in the best results, but it can be a useful method for understanding the network and identifying key connections. Weight pruning involves identifying connections with low weights and removing them from the network. This can be done using magnitude-based pruning and percentile-based pruning. Neuron pruning involves identifying and removing entire neurons that are not contributing to the performance of the network. This can be done using structural pruning and conditional pruning. Hence, pruning a YOLOv3 pre-trained model for COCO dataset helps to improve the performance and efficiency of the network by reducing the number of connections and computational requirements.

Quantization accelerates the DNN network and enables easy deployment of DNN models on resource-constrained devices. Furthermore, combining pruning and quantization reduces inference time as well as space complexity of the network by maintaining significant accuracy. A few existing approaches are quantizing every filter and activation layer of a pre-trained DNN using hierarchical deep reinforcement learning. In addition combining network pruning and quantization helps to produce a compact DNN model to fit into IoT devices for quick response. Furthermore, the quantization of weights of convolution layers of the trained model reduces the memory footprints and speeds up the model's inference. For example, the k-means clustering algorithm, followed by 8-bit quantization and integer-valued weights, makes the model compact and speeds up the inference. In addition, unique variants of the weight matrix are reduced by quantization.

Quantization enables a reduction in model size, which makes the model easy to deploy on hardware platforms. The Commonly used quantization approaches for reducing the model size are:K-means clustering based weight grouping, Binary quantization, One-bit/ two-bit

quantization, Binary convolution filter, Network weight mapping for trainable range, Binary or ternary weights quantization, 8-bit integer weights and activation.

In addition to pruning and quantization other methods are Knowledge distillation and Fused tile partitioning (FTP). Knowledge distillation is an approach to extract the knowledge from teacher (large DNN) to student DNN (small network) in a teacher-student architecture.

FTP is an approach for dividing convolutional layers into independent distributable tasks. Initially, layers are fused and vertically partitioned to minimize the footprint with reduced communication size and inference latency by avoiding centralized data synchronization.

Recent efforts towards reducing the computation and storage cost, a commonly used model compression approach prunes the network's parameters, creating a sparse network. In addition, L1 normalization makes the sparsified model speed up the computation. A general practice is either to prune the weights with small magnitudes and then retrain without hurting the overall accuracy; or prune the convolutional layers with an equal percentage of pruning rate followed by retraining shows the FLOP reduction without significant loss in original accuracy. Another approach is to prune the neurons rather than the parameters. These techniques require finding complete zero rows of weight matrices or nearly identical pairs of rows to prune a single neuron. Other methods to lower the convolutional overheads include FFT-based convolutions and fast convolution using the Winograd algorithm.

Table 3 depicts the state of the art of CNN based pruning and quantization approaches on various datasets and DNN models. Benoit et al. [32] proposed an integer-only arithmetic-based quantization scheme. Shaokai et al. [33] developed a DNN weight pruning and quantization framework. Author trainined iteratively with weight clustering and updated the centroid. It is observed that weight clustering and retraining improves the performance of the model. Geng Yuan et al. [34] proposed a memristor-based framework considering weight pruning and quantization. Muhamad et al. [35] used DeepLIFT for pruning DNN to prune filters and prune the weights of the fully convolutional layer. By using clustering-

Table 3 State of art of CNN based model compression

Ref	Dataset/Model	Key Performance Indicator(KPI)	Contribution
[32]	ImageNet/ COCO with MobileNet	latency-vs-accuracy	Proposed integer-only arithmetic based quantization scheme than floating point inference
[33]	MNIST Dataset with LeNet-5, ImageNet Dataset with AlexNet	Weight reduction of 167X in LeNet-5, in AlexNet 24.7X, and 23.4X in VGGNet, 1,910X and 210X respectively on memory reduction on LeNet-5 and AlexNet for weight data	DNN weight pruning and clustering/quantization framework for iterative weight clustering training and updating centroid, weight clustering retraining to improve the model's performance.
[34]	CIFAR-10 with ResNet18 and VGG-16 MNIST dataset with LeNet-5	29.81X, 20.88X weight compression ratio, with power reduction by 98.38%, 96.96% and 98.29%, 97.47% area reduction with 0.5%, 0.76% accuracy loss on VGG-16, ResNet-18 network respectively	A memristor-based framework is proposed considering weight pruning and quantization with alternating direction method of multipliers (ADMM) into DNN training.

Table 3 continued

Ref	Dataset/Model	Key Performance Indicator(KPI)	Contribution
[35]	Flowers 102 with Oxford Flowers Dataset ResNet56 on CIFAR10	Accuracy Vs percentage of parameters, Accuracy Vs number of bits	Used DeepLIFT for pruning DNN to prune filters and weights of the fully convolutional layer and followed by clustering-based quantization of DNN weights. Also, integer-based mixed-precision quantization for varying number of integer bits of each layer of DNN.
[12]	ImageNet with AlexNet/MobileNet-V1/ResNet-50	56.62% top-1 accuracy and 79.06% top-5 accuracy, compression rate 138.96X	Proposed one-shot pruning-quantization, which compresses through pre-trained weight parameters. During fine-tuning weight parameters are updated. Further, channel-wise quantization for each layer having common codebook still reduces bitrate.
[36]	Keyword Spotting Model/ 250K parameter	3X CPU consumption and 4x improvement in memory consumption	Proposed 8-bit quantization technique using tanh on dense layer weights followed by linear quantization on rest of the network.
[37]	CIFAR-10 dataset with ImageNet, and MNIST datasets with LeNet-5, AlexNet, and ResNet-50	Achieved overall weight pruning of 348X on LeNet-5 on AlexNet, 36X, and 8X on ResNet-50.	Proposed a joint weight pruning and quantization-based improved ADMM-NN framework.

based quantization of DNN weights author used integer-based mixed-precision quantization for varying numbers of integer bits for each layer of DNN. Peng Hu et al. [12] proposed one-shot pruning- quantization, which compresses the pre-trained weight parameters and further weights updates during fine-tuning. In addition, channel-wise quantization for each layer having a common codebook still reduces bitrate. Lu Zeng et al. [36] proposed an 8-bit quantization technique using tanh on dense layer weights followed by linear quantization on the rest of the network. Xiaolong Ma et al. [37] proposed a joint weight pruning and quantization based improved ADMM-NN framework.

2.2 Energy efficient CNN model

Most of the previous work focuses on reducing the number of weights and bit-width representations to reduce latency and memory footprint on object classification DNN models. However, in addition to the critical performance metrics, energy consumption is a big issue in deep neural network-based smart device-based applications.

More computation of deep neural network-based methods leads to high energy consumption. For example, object detection based on DNN requires 311X to 13,486X more energy than a traditional histogram of oriented gradients (HOG). Hence deploying DNN on an embedded or intelligent device is challenging where devices have limited energy or devices

operated with the battery. Table 4 describes the research papers worked on optimizing the energy efficiency.

SongHan et al. [20] developed a three-stage pipeline architecture with pruning, quantization, and Huffman coding for object recognition applications. Tien-Ju Yang et al. [38] proposed an energy-aware pruning algorithm for CNNs for object detection task with only pruning approach. Ruizhou Ding et al. [39] performed quantization and training to reduce computation energy and memory access. He showed that his proposed work, LightNN is hardware friendly considering accuracy and energy trade-off. Mi Sun Park et al. [40] proposed SQunatizer, an approach to jointly optimize both Sparse and low-precision NN. This method brings sparsification and low-bit quantization into a single training pass. The framework is extended for YOLOv3 for the image size 416X416 and achieved a compression rate of 17X compared with the baseline model. Yang Hua et al. [41] performed network pruning to 16 layers of YOLOv3 and 5-bit weight quantization with an mAP of 71.77%. Author [42] proposed designing energy-efficient deep neural networks through post-training quantization without retraining.

Our work focuses on optimizing the pre-trained model through filter pruning followed by 8-bit quantization on the YOLOv3 object detection model on the COCO2017 data set. We evaluated the model with the Inference latency, Number of parameters, mean Average Precision, and Energy Efficiency.

Table 4 State of art of design of energy efficient CNN model

Ref	Dataset/Model	Contribution	KPI
[20]	ImageNet dataset/VGG-16	Pruning+ Quantization + Huffman coding	Reduces the number of connections by upto 13X and reduction in number of bits from 32 to 5 bit with Quantization, reduces storage by 240MB to 6.9MB (AlexNet), storage from 552MB to 11.3MB (VGG-16), upto 4X speedup layer wise and **improved energy consumption of 3X to 7X**
[38]	ImageNet ILSVRC 2014 dataset/AlexNet and GoogLeNet	Pruning	**Energy consumption** of AlexNet reduced by 3.7X and GoogLeNet is reduced by 1.6X, with less than 1% top-5 accuracy loss.
[39]	MNIST, CIFAR-10 dataset/DNN, BNN	Quantization	Reported various **accuracy and energy** trade-off options.
[40]	ImageNet ILSVRC2012 dataset/ResNet18, ResNet50, MobileNet-v2. COCO dataset/ YOLOv3	Low-bit quantization	**Optimization-** of sparse and low-precision NN- Compression rates of 18X, 17X, 9X respectively for Resnet 18, 50, MobileNet-v2. SQuantization on YOLOv3 achieved 17X compression rate for 4 bit.
[41]	PascalVoC dataset/ YOLOv3	Pruning + Weight quantization	**Energy efficiency** with mAP of 76.48% of int 8, 77.09% of Float32.
[42]	ImageNet dataset / ResNet-18, 34, 50, 152 model	8-bit Quantization	Quantized DNN results **27 X energy reduction** with reference to baseline model.
Our work	COCO2017 dataset/ YOLOv3	Pruning without fine tuning + 8-bit quantization	Focus on Inference latency, Number of parameters, mean Average Precision, **Energy Efficiency**.

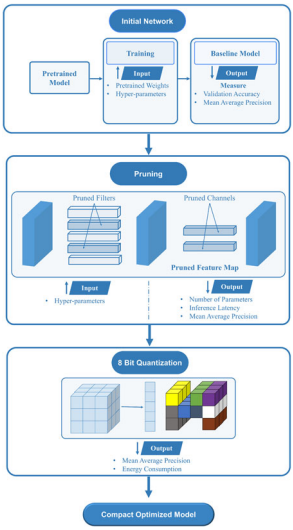


Fig. 2 Proposed model: We optimize the pre-trained YOLOv3-SPP, a real-time object detection model. Pruning and 8-bit quantization are applied to reduce the model size and computational latency

Therefore to design energy-efficient DNNs for realizing in small embedded devices, many researchers work on either algorithmic or hardware architectural perspective. From the algorithmic perspective, the main focus is on reducing the model's number of parameters and bit-width. Convolutional layers dominate energy consumption in a DNN due to the feature map and data movement.

3 Proposed system

Energy consumption includes computation energy consumption and data movement energy consumption. Our goal is to reduce the computation energy consumption by designing the optimized compact model through pruning and quantization. The proposed model consists of three modules such as initial training to develop baseline model, Pruning, and followed by Quantization as shown in Fig. 2.

Overview of software and framework used in the article is shown in Table 5. Our focus is highlighted.

Table 5 Our considerations for designing proposed framework

Aspects	Various aspects (our consideration is in bold)
Optimization	**Network Optimization**, Knowledge distillation, Network architecture
Network Optimization	Factorization, **Pruning, Quantization**
Hardware Acceleration Platform	**CPU, GPU**, Application-Specific Integrated Circuit (ASIC), Field Programmable Gate Arrays (FPGA)
Metrics	Computation time, Memory optimization, **Energy Efficiency**

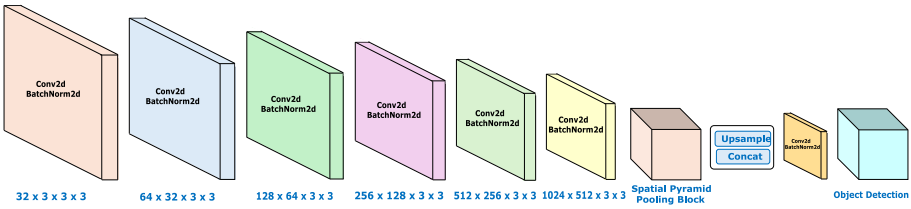


Fig. 3 Layered architecture of YOLOv3-SPP model for object detection system

3.1 Training

We consider the prtrained YOLOv3-Spatial Pyramid Pooling(SPP) model configuration file, pretrained weights and COCO2017 dataset. Initiailly we train the YOLOv3 network with default configuration such as batch size, epochs and considering the image size of 416 X 416 and 608 X 608 image.

Figure 3 depicts the YOLOv3-SPP layered architecture used for object detection. YOLOv3-SPP model is built on YOLOv3 architecture with addition module SPP for improving the feature extraction. It Supports image of size 416X416, 608X 608. Darknet-53 is used as a backbone for feature extraction. SPP module enables to extract multi-scale features on series of pooling operations. Hence SPP improves the detection of objects of different sizes. Convolutional layers extract features and predict bounding boxes, objectness scores and class probabilities. SPP layer has been added before the detection module to pool the feature map using varied size pooling windows, followed by concatenating results into single feature vector. This enables capturing spatial features at multiple scales. Input to YOLOv3-SPP model is of size 416X416 or 608X608. Darknet-53 network act as a backbone for extracting feature map using series of convolutional layer. As shown in the figure, after feature map goes to SPP layer. SPP applies maxpooling at different sizes and pooled feature maps are concatenated into single output. YOLOv3-SPP makes predictions using three different scales such as bounding box coordinates, objectness score, class probabilities.

3.2 Pruning

The next step is to prune the model with a sparsity value of 0.1 (between 0 and 1). The larger the value the data will be too sparse; which increases the complexity of the model. After pruning, we calculated the mean average precision of the network before and after pruning. The complete framework is implemented in python with torch version 1.12.1+cu113, OpenCV, NumPy 1.21.6, and tqdm.

Algorithm 1 describes the pruning process. The first module takes pre-trained model, weight and input image, hyper-parameters and pruning percentage values as inputs. Followed by based on the number of epochs, training module executes the process for the batch of data and prunes the channels based on the threshold value set. After pruning, we calculated the model's mAP and the number of parameters. Here pruning is performed without retraining.

3.3 Quantization

Quantization reduces the model size and hence makes the model easy to deploy on hardware platforms. The different ways to perform quantization are grouping the weights using k-

Algorithm 1 Pruning the pre-trained model, dataset.

Input: Pre-trained model, pre-trained weight
Output: Optimized sub model
Initialization: Obtain Pre-trained model, pre-trained weights,source data
Set Learning_rate=0.1, batchsize=64, momentum=0.9, no_of_epochs=100
Pruning(channel pruning percentage p=0%(in the begining))
Execute training on pruned model
for $i \leftarrow 0$ to no_of_epochs **do**
    Batch of data from pre-training data
    Set prune percentage
    Set threshold pruning
    prune based on threshold value
end for
Obtain the mean accuracy precision (mAP) of the model
Execute test against the baseline YOLOv3-SPP
Find the difference with number of parameters
return compact optimized model (.pt model)

means clustering, binary quantization, one-bit/ two-bit quantization, binary convolution filter, network weight mapping for trainable range, binary or ternary weights quantization, 8-bit integer-weights and activation for reducing the model size.

We optimize the model using integer-based uniform quantization. This speeds up inference and reduces the memory requirements. Instead of reducing the bit-precision one by one, we collectively reduce the bit-precision in all layers, according to the sensitivities of the layers. When accuracy goes below a certain threshold, we stop and start the iterations. Proposed method is implemented using PyTorch. We set the batch size 32 for the model at training stage. The SDG optimizer compresses the model with the momentum of 0.9, weight decay of 10-4 and learning rate of 0.005.

Algorithm 2 clearly depicts the process of quantization. Pruned pre-trained model from Algorithm 1, number of quantization bits and hyper parameters are the input to Algorithm 2. In 8-bit quantization, k-means clustering helps to group the weights of the CNN into a fixed number of clusters, each representing a unique range of values. The k-means clustering process involves selecting a fixed number of clusters (k) and randomly initializing the centroids

Algorithm 2 Optimizing the model through Quantization.

Input: A pruned pre-trained model with L layers, quantization bit width b, batch size bs and epoch number E
Output: Compressed Model
Quantization (Pre-trained Model, set target bit-width)
Initialization : Obtain pruned model, weights,data
Set Quantization_bit_width b=8, batch_size bs=64, no_of_epochs=10
For the layers of the model, perform quantization on conv2d
Execute k-means clustering on pruned weights of the layers
for $i \leftarrow 1$ to no_of_epochs **do**
repeat
    Obtain different clusters and centroids
    Replace weights with centroid value for every cluster
until
    all the samples selected
end for
Train with 8-bit quantization module on updated weights
Assign the updated weights to the layer
Find the accuracy of optimized model
return compressed model

of those clusters. Next, the data is grouped into a specified number of clusters based on nearest centroid. The centroids are then updated to the mean of the data points in their respective clusters, and the process is repeated until convergence. In 8-bit quantization, each cluster represents a range of values with 8 bits. The weights of the CNN are then quantized to the nearest cluster centroid, resulting in fewer bits for each weight representation. Hence quantization can significantly reduce the memory and computational requirements of the CNN, making it more efficient and easier to deploy on devices with limited resources.

4 Experimental setup and Result

4.1 Dataset

COCO is a publicly available object detection segmentation and captioning dataset [44]. It consists of 330K images having 80 object categories with more than 200K labelled images with training, validation, test sets and annotations on training and validation sets. COCO2017 consists of 118k files in the train, 40.7k files in the test, and 5000 files for validation and consists of annotations files. In our framework, we consider YOLOv3-SPP, a pre-trained model with pre-trained weights for training. The YOLOv3-SPP model has a better feature in max-pooling layers and downsampling in convolutional layers with stride 2 with higher accuracy compared to YOLOv3 and YOLOv3-tiny. This article considers YOLOv3-SPP a pre-trained model trained with the COCO dataset. The pre-trained model [43] and its weights are downloaded and fed into the framework.

4.2 Experimental setup

We used Nvidia V100 Tensor Core GPUs and Intel i5 8th generation CPU for the model training and testing. For the implementation, we used Python programming language, PyTorch for deep learning model, NumPy for array manipulation, Matplotlib for visualization, and scikit-learn for K-means clustering.

4.3 Results

4.3.1 Baseline model

We consider YOLOv3-SPP, a pre-trained model with pre-trained weights for training. We train the model on the COCO2017 dataset, batch size, image size and default configuration parameters to develop a baseline model. The initial pre-trained model configuration file, index files, and categories trained for the hyperparameters are as shown in Table 6. With this, we analyze the model performance before pruning and quantization.

Darknet is a C and CUDA based opens source framework to train neural network and act as backbone for YOLO. Darknet enables YOLO training and supports both CPU and GPU computation. Darknet achieves significantly higher FLOPs also supports GPU for better, efficient, and faster computation. We have set the model fitness function to a weighted combination of metrics: mAP@0.5 contributes 10% of the weight with initial recall and precision set to zero. YOLOv3-SPP is proved better than the original YOLOv3 on the COCO dataset, with significant high detection accuracy. SPP is most successful in having flexibility

Table 6 Model hyperparameters to develop baseline model

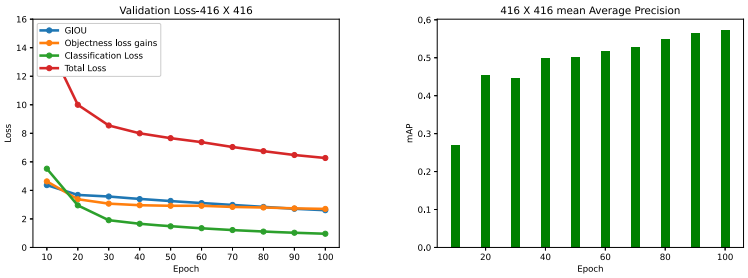
Parameters	Values
Data	Coco2017
Batch size	16
Model	Yolov3-spp
Weights	Yolov3-spp. weights
Configuration	Yolov3-spp.cfg
Data loader workers	4
Image size	416*416 or 608*608
Epochs	100
Model summary	
Layers	225
parameters	6.2998e+07
Gradients	6.2998e+07
Optimizer groups	
Bias	76
Conv2d.weight	76
other	76

in feeding input scales and weights during training in computer vision applications with improved performance. Hence, we considered YOLOv3-SPP for the object detection system in our experiment.

4.3.2 Training the model

The training process consists of data augmentation for cropping, padding, rotation, height, and width for feeding the input data into a neural network. Annotation is performed to convert the label into YOLO format. As shown in Table 6, training is conducted on image size with width 416 and height 416 and also for 608 X 608. Hence, every image will be resized to this size during training and detection. Training is performed for 100 epochs with a final learning rate of 0.0005. Figure 4 a) shows validation loss for the input image of size 416 X 416 for 100 epochs. Initial Validation loss was 14.5 and started decreasing up to 6.27 as the number of epochs increased. Figure 4 b) indicates the mAP0.5, incrementing from 26.9% to 57.3% for 100 epochs. Figure 5 a) shows the validation loss for the image size of 608 X 608 for 100 epochs. As we can see from Figure 5 b), validation loss started decrementing from 12.7 to 4.95 and mean average precision gradually increased from 30.3% to 61%. Compared to Figs. 4 a) and 5 a), validation loss was reduced more to 608 X 608 size image than 416 X 416 image. Comparative analysis of Figs. 4 b) and 5 b) shows that the 608 X 608 image has high mean average precision.

Training is performed for 100 epochs on the COCO2017 dataset with an input image size of 416X416 and 608 X 608. The Figs. 4 a) and 5 a), shows that validation loss reduces for a larger model to improve the performance, and mAP increases as the number of epochs increases as shown in Figs. 4 b) and 5 b). Baseline model results with inference speed of 22ms and 43ms for 416*416 and 608*608, respectively.



(a) (b)

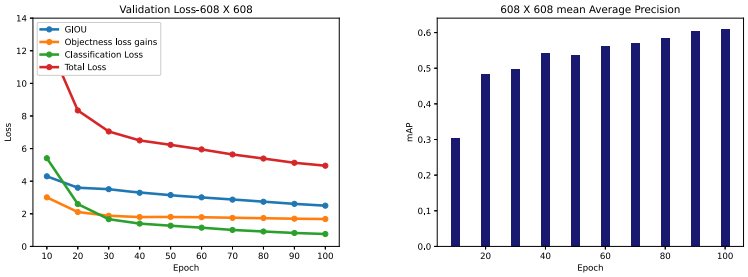
Fig. 4 (a) Validation loss (Baseline model) for 100 epochs for an image size of 416 X 416. The graph depicts a reduction in validation loss as the number of epochs increases (b) Mean Average Precision (Baseline model) for 100 epochs for an image size of 416 X 416. The graph depicts a gradual increment in mAP as the number of epochs increases

Figure 6 a) depicts the training loss and testing loss for 100 epochs for the image size of 416 X 416. Fig. 6 b) depicts the training loss and testing loss for 100 epochs for the image size of 608 X 608. These values indicates the loss functions of training and testing phases. As number of epochs increased both training loss started decreasing as model learnt well during the training phase. Training loss stabilizes after more number of epochs as model learns. Here training loss indicates the total loss considering localization loss, objectness loss and classification loss. Initially testing loss was high, but after convergence testing loss decreses and finally stablizes for more number of epochs.

4.3.3 Pruning the model

The pruning process starts with a pre-trained DNN model. Proposed model derived based on YOLOv3, with 416X416 input size, and train on COCO dataset.

Pre-trained model is pruned to get optimized model with reduced number of parameters while maintaining the significant accuracy. Reweighted regularization pruning algorithm, the pruned model can be obtained.



(a) (b)

Fig. 5 (a) Validation loss (Baseline model) for 100 epochs for an image size of 608 X 608. The graph depicts a reduction in validation loss as the number of epochs increases (b) Mean Average Precision (Baseline model) for 100 epochs for an image size of 608 X 608. The graph depicts a gradual increment in mAP as the number of epochs increases



(a) (b)

Fig. 6 (a) Training loss and Testing loss 100 epochs for an image size of 416 X 416. (a) Training loss and Testing loss 100 epochs for an image size of 608 X 608

Figure 7 shows channels before and after pruning. Pruning is performed on a pre-trained model for YOLOv3-SPP with the COCO2017 dataset with an 80% of pruning ratio. Consider a threshold value for the parameters of the layers. The parameter values less than the threshold value are set to zero, which helps to reduce the number of parameters significantly. As a result, the model size after pruning and sparsification are smaller than the size of the original model. Total number of channels reduced from 13376 to 3535 with the pruning ratio of 73.6%. Experiment results parameters reduced from 62.998749M to 12.350941M with inference latency reduced from 0.2401 to 0.1483 with increased mAP@0.5 value of 61.85% compared to 61% mAP@0.5 before pruning.

Table 7 shows change in number of channels before and after pruning of layers of the model. We observe number of channels are greatly reduced for few layers.

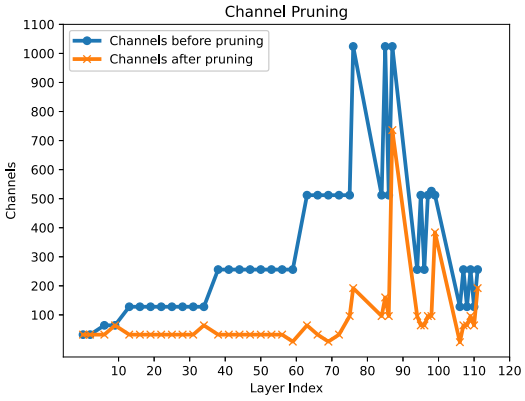
We calculated various performance metrics of the model with the following equations. The average mAP is calculated with the following equation:

$$maP = \int_0^1 p(x)d(x) \tag{1}$$

where p(x) is the precision of the object detection

$$Precision = \frac{TP}{TP + FP} \tag{2}$$

Fig. 7 Channel Pruning-Pruning is performed on a pre-trained model for YOLOv3-SPP with the COCO2017 dataset with an 80% of pruning ratio



$$Recall = \frac{TP}{TP + FN} \tag{3}$$

where TP represents true positives, FP indicates false positives, FN indicates false negative.

$$F1 = \frac{2XPrecisionXRecall}{Precision + Recall} \tag{4}$$

F1 score is measure of test accuracy. FLOPS (Floating point operations) is used to measure the model computational complexity. For convolutional layers:

$$FLOPS = 2HW(C_{in}k^2 + 1)C_{out} \tag{5}$$

H and W are input feature map. K is the size of kernel. C_{in} and C_{out} are input and output channels.

For fully connected Layer:

$$FLOPS = (2C_{in} - 1)C_{out} \tag{6}$$

where C_{in} and C_{out} are input and output channels.

Acceleration Ratio (AR) indicates time taken for inference by the pruned model with reference to the original model.

$$AccelerationRatio = \frac{Time_o}{Time_p} \tag{7}$$

Where $Time_o$ and $Time_p$ represents the inference time of the original model and inference time of the pruned model respectively.

Pruning is performed on pretrained model for YOLOv3-SPP with COCO2017 dataset with 80% of pruning ratio. We achieved precision 0.6970, Recall 0.74, mAP@0.5 0.6185 and F1 score 0.7178 as represented in Table 8. Precision, recall and F1 score increases for different pruning percentage. For 80% of pruning, model achieved a precision of 69.70%, which indicates model correctly predicts the positive class out of total number of predictions. Recall of 0.74 indicates 74% of objects present in the image are being detected by the model and considered to be good model. F1 score balances the precision and recall values and hence act as a better parameter for model evaluation. 71.78% of F1 score signifies balanced performance and good trade-off between precision and recall. The mAP measures how well model detects the objects of a class and 61.85% indicates model is effective in detecting the objects. Channels with Gamma value less than 1.0462 are pruned and threshold should be

Table 7 Number of channel before and after Pruning

Layer	Channel before	Channel after
0	32	18
3	32	23
6	64	25
9	64	37
13	128	45
18	128	38
41	256	47
53	512	56
78	1024	123

Table 8 Pruning percentage and corresponding mAP

Pruning (%)	mAP@0.5	Precision	Recall	F1
0	0.61	0.675	0.73	0.701423
70	0.608	0.669	0.745	0.7049
75	0.6156	0.6598	0.769	0.7102
80	0.6185	0.6970	0.74	0.7178

less than 0.9900. Number of channels reduced from 13376 to 3536 with the actual pruning ration of 0.736.

As shown in Table 9, after pruning we evaluated the model with mAP and number of parameters. Pruning is performed with 0%, 70%,75%, and 80% of pruning ratio. With the 80% of pruning ratio, achieved 61.85% of mAP with prediction time 17ms after pruning.

4.3.4 Quantization

To reduce the computing, data movement and consequently to lower the energy usage we performed the pruning followed by quantization by representing data in 8-bit. This reduces number of bits from 32 to 8 bits. The evaluation of our approach when network architecture uniformly reduces to 8-bit network precision through quantization. After the quantization model achieved 61.86% mAP.

4.3.5 Comparative analysis

As shown in Table 10, SQuantization [40] is applied except first layer of YOLOv3 with Darknet-53, on COCO dataset having 80 classes with the image size 416 X 416 pixels. Our results are compared with SQuantization results. As shown in Table 10, 4 bit SQuantizer gives 50.3 mAP compared to baseline mAP 52.8 with slight lesser than the orginal mAP value. In our work, Pruning and quantization is applied for YOLOv3-SPP on COCO dataset, and achieved significantly greater mAP (61.8676%) compared to the baseline model.

For distributed edge device computation, inference latency, energy consumption and communication overhead depend on the number of parameters in the model. Pruning, followed by 8-bit quantization, reduces computational resources and power consumption by a Finite number of bits of representation for continuous values. As in the quantization process, a 32-bit floating point number converts to an 8-bit integer, and k-means cluster-based quantization reduces model size, inference latency to 17ms and reduction in energy consumption by 29.41%. Hence makes the model efficient, easy to deploy on hardware platforms for intelligent application processing.

Table 9 KPIs: mAP, Inference /prediction time, parameters before & after pruning

Metric	Before	After
mAP	0.702388	0.7
Parameters	62998749	12350941
Inference	0.2401	0.1483

Table 10 Comparison of our work with existing SQuantizer

Reference	Work	mAP	Parameters
Baseline in [40]	Before SQuantization	52.8	61.95
Baseline in [40]	After SQuantization	50.3	33.63
Baseline of our work	Before P+Q	61	62
Baseline of our work	After P+Q	61.8676	12.35

P+Q stands for Pruning and Quantization
Compared to [40], the Energy consumed in our work is 29.41% lesser

5 Conclusion

Pruning and quantization are the techniques used to reduce the computational complexity and energy consumption of neural network models, particularly useful when deployed on edge devices with limited resources. Pruning involves removing unnecessary connections and parameters from the model, which reduces the number of calculations that need to be performed during inference. Quantization reduces the precision of the model's parameters and activations from 32-bit floating point values to lower-precision integers. By combining pruning and quantization, it is possible further to reduce the model's computational complexity and energy consumption.

We have considered the YOLOv3-SPP pre-trained object detection network on a large-scale COCO2017 dataset. To accelerate the CNN model, we prune the model by removing unimportant weights, followed by quantization to generate the sub-optimized model of the pre-trained model. Evaluation of our sub-optimized model shows that pruning reduces model parameters from 62998749 to 12350941, with 61.85% of mAP@0.5 and inference latency of 17ms. The combination of pruning and 8-bit quantization results in a mean average precision of 61.8676%, an improvement in inference latency by 22.72%, and a reduction of energy consumption by 29.41%. The work demonstrates that combining these two techniques can yield significant energy savings without sacrificing accuracy. Furthermore, we show the optimal combination of pruning and quantization parameters to achieve the best energy efficiency. Overall, this work highlights the potential for pruning and quantization to optimize the energy consumption of deep learning models at the edge.

The future scope involves the development of new hybrid pruning and quantization algorithms that are specifically designed to optimize energy consumption at edge devices. These algorithms could combine the best features of pruning and quantization, leveraging the strengths of both approaches to achieve greater energy efficiency. The trade-offs between model size, accuracy, computational efficiency will be studied in future to understand the impact of layer-wise pruning and quantization. There is a massive opportunity for researchers and developers to continue exploring and refining these techniques to improve the performance and efficiency of edge computing systems.

Funding Open access funding provided by Manipal Academy of Higher Education, Manipal. This research did not receive any specific funding.

Data Availability The datasets considered during the current study are publically available in the repository https://cocodataset.org/.

Declarations

Ethics approval Not Applicable.

Conflict of interest The authors declare no conflict of interest.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit http://creativecommons.org/licenses/by/4.0/.

References

1. Wang X, Han Y, Leung VCM, Niyato D, Yan X, Chen X (2020) Convergence of edge computing and deep learning: A comprehensive survey. IEEE Commun Surv Tutor 22(2):869–904. https://doi.org/10.1109/COMST.2020.2970550
2. Mukherje D, Anand A (2023) On edge deep learning implementation: approach to achieve 5G. Multimed Tools Appl 82:12229-12243. https://doi.org/10.1007/s11042-022-13712-3
3. Naveen S, Kounte MR (2022) Distributing the Cloud into Fog and Edge: New Weather in IoT Based Deep Learning. In: Gunjan VK, Zurada JM (eds) Proceedings of the 2nd International Conference on Recent Trends in Machine Learning, IoT, Smart Cities and Applications. Lecture Notes in Networks and Systems, vol 237. Springer, Singapore, pp 749-758. https://doi.org/10.1007/978-981-16-6407-6-64
4. Yang CY, Lin YN, Wang SK et al (2024) An Edge Computing System for Fast Image Recognition Based on Convolutional Neural Network and Petri Net Model. Multimed Tools Appl 83:12849-12873. https://doi.org/10.1007/s11042-023-15388-9
5. Parashar J, Kushwah VS, Rai M (2023) Determination Human Behavior Prediction Supported by Cognitive Computing-Based Neural Network. In: Kumar R, Verma AK, Sharma TK, Verma OP, Sharma S (eds) Soft Computing: Theories and Applications. Lecture Notes in Networks and Systems, vol 627. Springer, Singapore. https://doi.org/10.1007/978-981-19-9858-4_36
6. Brar DS, Aggarwal AK, Nanda V, Kaur S, Saxena S, Gautam S (2024) Detection of sugar syrup adulteration in unifloral honey using deep learning framework: an effective quality analysis technique. Food and Humanity, vol 2, 100190. https://doi.org/10.1016/j.foohum.2023.11.017
7. Maini DS, Aggarwal AK (2018) Camera position estimation using 2D image dataset. Int J Innov Eng Technol 10(2):199-203. http://dx.doi.org/10.21172/ijiet.102.29
8. Aggarwal AK (2023) A Review on Genomics Data Analysis using Machine Learning. WSEAS Trans Biol Biomed 20:119-131. https://doi.org/10.37394/23208.2023.20.12
9. Aggarwal AK (2015) A hybrid approach to GPS improvement in urban canyons? Int J Eng Sci Res Technol 4(10):358–363
10. Naveen S, Kounte MR (2022) Memory optimization at edge for distributed convolution neural network. Trans Emerg Telecommun Technol 33(12), https://doi.org/10.1002/ett.4648
11. Liang T, Glossner J, Wang L, Shi S, Zhang X (2021) Pruning and quantization for deep neural network acceleration: A survey. Neurocomput 461:370-403. https://doi.org/10.1016/j.neucom.2021.07.045
12. Hu P, Peng X, Zhu H, Aly MMS, Lin J (2021) OPQ: Compressing Deep Neural Networks with One-shot Pruning-Quantization. In: Proceedings of the AAAI Conference on Artificial Intelligence 35(9):7780-7788. https://doi.org/10.1609/aaai.v35i9.16950
13. Xu W, Fang W, Ding Y, Zou M, Xiong N (2021) Accelerating federated learning for iot in big data analytics with pruning, quantization and selective updating. IEEE Access 9:38457–38466. https://doi.org/10.1109/ACCESS.2021.3063291
14. Park JH, Kim KM, Lee S (2022) Quantized Sparse Training: A Unified Trainable Framework for Joint Pruning and Quantization in DNNs. ACM Trans Embed Comput Syst (TECS) 21(5):1-22. https://doi.org/10.1145/3524066

15. Russo E, Palesi M, Monteleone S, Patti D, Mineo A, Ascia G, Catania V (2021) DNN model compression for IoT domain-specific hardware accelerators. IEEE Internet of Things J 9(9):6650–6662. https://doi.org/10.1109/JIOT.2021.3111723
16. Wang K, Liu M, Ye, Z (2021) An advanced YOLOv3 method for small-scale road object detection. Appl Soft Comput 112:107846. https://doi.org/10.1016/j.asoc.2021.107846
17. Adiono T, Putra A, Sutisna N, Syafalni I, Mulyawan R (2021) Low latency YOLOv3-tiny accelerator for low-cost FPGA using general matrix multiplication principle. IEEE access 9:141890–141913. https://doi.org/10.1109/ACCESS.2021.3120629
18. Jalali F, Hinton K, Ayre R, Alpcan T, Tucker RS (2016) Fog computing may help to save energy in cloud computing. IEEE J Select Areas Commun 34(5):1728–1739. https://doi.org/10.1109/JSAC.2016.2545559
19. Han S, Pool J, Tran J, Dally W (2015) Learning both weights and connections for efficient neural network. Adv Neural Inf Process Syst 28
20. Han S, Mao H, Dally WJ (2015) Deep compression: Compressing deep neural networks with pruning, trained quantization and huffman coding. arXiv preprint arXiv: https://doi.org/10.48550/arXiv.1510.00149
21. Naveen S, Kounte MR, Ahmed MR (2021) Low latency deep learning inference model for distributed intelligent IoT edge clusters. IEEE Access 9:160607–160621. https://doi.org/10.1109/ACCESS.2021.3131396
22. Deng S, Zhao H, Fang W, Yin J, Dustdar S, Zomaya AY (2020) Edge intelligence: The confluence of edge computing and artificial intelligence. IEEE Internet of Things J 7(8):7457–7469. https://doi.org/10.1109/JIOT.2020.2984887
23. Molchanov P, Tyree S, Karras T, Aila T, Kautz J (2016) Pruning convolutional neural networks for resource efficient inference. arXiv preprint arXiv:1611.06440
24. Min C, Wang A, Chen Y, Xu W, Chen X (2018) 2pfpce: Two-phase filter pruning based on conditional entropy. arXiv preprint arXiv:1809.02220
25. He Y, Liu P, Wang Z, Hu Z, Yang Y (2019) Filter pruning via geometric median for deep convolutional neural networks acceleration. In Proceedings of the IEEE/CVF conference on computer vision and pattern recognition (pp 4340-4349)
26. Polino A, Pascanu R, Alistarh D (2018) Model compression via distillation and quantization. arXiv preprint arXiv:1802.05668
27. Duan W, Liu Z, Jia C, Wang S, Ma S, Gao W (2022) Differential Weight Quantization For Multi-Model Compression. IEEE Trans Multimed 25:6397-6410. https://doi.org/10.1109/TMM.2022.3208530
28. Kim J, Chang S, Kwak N (2021) PQK: model compression via pruning, quantization, and knowledge distillation. arXiv preprint arXiv:2106.14681
29. Abbasi S, Hajabdollahi M, Karimi N, Samavi S (2020) Modeling teacher-student techniques in deep neural networks for knowledge distillation. In 2020 International Conference on Machine Vision and Image Processing (MVIP). IEEE 1-6. https://doi.org/10.1109/MVIP49855.2020.9116923
30. Sepahvand M, Abdali-Mohammadi F, Taherkordi A (2022) Teacher?student knowledge distillation based on decomposed deep feature representation for intelligent mobile applications. Expert Syst Appl 202:117474. https://doi.org/10.1016/j.eswa.2022.117474
31. Zhao Z, Barijough KM, Gerstlauer A (2018) Deepthings: Distributed adaptive deep learning inference on resource-constrained iot edge clusters. IEEE Trans Comput-Aided Des Integr Circ Syst 37(11):2348–2359. https://doi.org/10.1109/TCAD.2018.2858384
32. Jacob B, Kligys S, Chen B, Zhu M, Tang M, Howard A, Kalenichenko D (2018) Quantization and training of neural networks for efficient integer-arithmetic-only inference. In Proceedings of the IEEE conference on computer vision and pattern recognition (pp 2704-2713). https://doi.org/10.1109/CVPR.2018.00286
33. Ye S, Zhang T, Zhang K, Li J, Xie J, Liang Y,Wang Y (2018) A unified framework of dnn weight pruning and weight clustering/quantization using admm. arXiv preprint arXiv:1811.01907
34. Yuan G, Ma X, Ding C, Lin S, Zhang T, Jalali ZS, Wang Y (2019) An ultra-efficient memristor-based DNN framework with structured weight pruning and quantization using ADMM. In: (2019) IEEE/ACM International Symposium on Low Power Electronics and Design (ISLPED) (pp 1–6). IEEE. https://doi.org/10.1109/ISLPED.2019.8824944
35. Sabih M, Hannig F, Teich J (2020) Utilizing explainable AI for quantization and pruning of deep neural networks. arXiv preprint arXiv:2008.09072
36. Zeng L et al (2022) Sub 8-Bit Quantization of Streaming Keyword Spotting Models for Embedded Chipsets. In: Sojka P, Horak A, Kopecek I, Pala K (eds) Text, Speech, and Dialogue. TSD 2022. Lecture Notes in Computer Science, vol 13502. Springer, Cham. https://doi.org/10.1007/978-3-031-16270-1-30

37. Ma X, Lin S, Ye S, He Z, Zhang L, Yuan G, Wang Y (2021) Non-structured DNN weight pruning?Is it beneficial in any platform? IEEE Trans Neural Netw Learn Syst 33(9):4930–4944. https://doi.org/10.1109/TNNLS.2021.3063265
38. Yang TJ, Chen YH, Sze V (2017) Designing energy-efficient convolutional neural networks using energy-aware pruning. In: IEEE Conference on Computer Vision and Pattern Recognition (CVPR), Honolulu, HI, USA, 2017, pp. 6071-6079. https://doi.org/10.1109/CVPR.2017.643
39. Ding R, Liu Z, Chin TW, Marculescu D, Blanton RD (2019) Flightnns: Lightweight quantized deep neural networks for fast and accurate inference. In Proceedings of the 56th Annual Design Automation Conference 2019 (pp 1-6)
40. Park MS, Xu X, Brick C (2018) Squantizer: Simultaneous learning for both sparse and low-precision neural networks. arXiv preprint arXiv:1812.08301
41. Hua Y, Yu L, Meng X, Qin Z (2021) Quantize YOLOv3-tiny For 5-bit Hardware. In 2021 4th International Conference on Advanced Electronic Materials, Computers and Software Engineering (AEMCSE). IEEE 1047-1050. https://doi.org/10.1109/AEMCSE51986.2021.00214
42. Latotzke C, Balim B, Gemmeke T (2022) Post-Training Quantization for Energy Efficient Realization of Deep Neural Networks. In 2022 21st IEEE International Conference on Machine Learning and Applications (ICMLA). IEEE 1559-1566. https://doi.org/10.1109/ICMLA55696.2022.00243
43. YOLO pretrained model, [online] Available: https://pjreddie.com/darknet/yolo/ Accessed 11 July 2022
44. COCO dataset, [online] Available: https://cocodataset.org/ Accessed 11 July 2022

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.



Soumyalatha Naveen (Member, IEEE) received the Bachelor's degree in computer science and engineering and Master's degree from Visvesvaraya Technological University, Belgaum, India, and the Ph.D. degree in the domain of Edge Intelligence from REVA University, Bengaluru. She is presently serving as Assistant professor- Senior Scale in the department of Computer Science and Engineering at Manipal Institute of Technology Bengaluru, Manipal Academy of Higher Education. Prior to this she was associated with REVA University, Brindavan College of Engineering and was a software engineer. She has more than 12 years of teaching, research, and administrative experience. She has published over 8+ Indexed Journal papers, 25+ International Conference papers and 3 book chapters. Her research areas include Artificial intelligence, Deep Learning, Edge computing, Internet of Things, Intelligent IoT systems.



Manjunath R Kounte is currently working as Professor and Head, Electronics and Computer Engineering, REVA University, Bengaluru, India. He has over 16+ years of Teaching, Research and Administrative experience. He has completed his Bachelors in Electronics and Communication Engineering and Master's in Computer Network Engineering from VTU Belagavi, India and PhD in Computational Cognitive Neuroscience from Jain University, Bengaluru, India. Manjunath R Kounte is Senior Member, IEEE, and member of various international professional bodies such as IETE, ISTE. He has over published over 20+ Indexed Journal papers, 30+ International Conference papers, 2 books and 7 book chapters. He has 2 International Patents Granted to his Credit, 1 Design Patent Granted and 6 Indian Patents Published. His overall publications have reached citations with over 500+ in Scopus and h-index of 11. He has successfully guided 4 research scholars for award of PhD and currently 5 scholars are pursuing PhD under his guidance. His research interests include Machine Learning, IoT Edge Computing, Internet of Vehicles, Signal Processing, Wireless Communication, Blockchain Technologies etc. He is also heading Machine Learning and Blockchain (MLB) research lab in REVA University.